SkillSwap

Software Requirements & System Design

Technical Specification Document v1.0

Peer-to-Peer Skill Exchange Platform

April 2026

Table of Contents

Right-click the TOC and select "Update Field" to refresh page numbers

Table of Contents **2**

1. Introduction **6**

1.1 Purpose of the Platform 6

1.2 Scope of the System 6

1.3 Target Users 6

1.4 Definitions and Terminology 7

2. Problem Statement **8**

2.1 Real-World Problems 8

Expensive Learning Platforms 8

Lack of Peer-to-Peer Knowledge Exchange 8

Limited Accessibility to Skill Learning 8

Need for Collaborative Learning Ecosystems 8

2.2 How SkillSwap Solves These Problems 8

3. System Overview **10**

3.1 Platform Architecture 10

3.2 Major System Modules 10

Authentication System 10

User Profile Module 10

Skill Management Module 10

Skill Discovery Module 10

Exchange Request System 10

Chat System 10

Session Scheduling Module 10

Review & Rating System 11

Notification System 11

4. Functional Requirements **12**

4.1 User Management 12

4.2 Skill Management 12

4.3 Search & Filtering 13

4.4 Exchange Requests 13
4.5 Session Scheduling 14
4.6 Messaging 14
4.7 Notifications 15
4.8 Ratings & Reviews 15

5. Non-Functional Requirements 16
5.1 Scalability 16
5.2 Availability 16
5.3 Performance 16
5.4 Reliability 17
5.5 Security 17
5.6 Maintainability 17

6. Technology Stack 18
6.1 Frontend Stack 18
6.2 Backend Stack 18
6.3 Database & Storage 19
6.4 Infrastructure 19
6.5 Technology Selection Rationale 19

7. High-Level System Architecture 21
7.1 Architecture Overview 21
7.2 Component Description 21
Client Layer 21
API Gateway 21
Microservices Layer 22
Data Layer 22
7.3 Data Flow 22

8. Database Design 24
8.1 Database Selection 24
8.2 Collection Schema 24
Users Collection 24
Skills Collection 24
SkillRequests Collection 24
Sessions Collection 24
Messages Collection 24

Reviews Collection 24
Notifications Collection 25
8.3 Database Indexes 25
9. API Design 27
9.1 API Overview 27
9.2 Authentication APIs 27
9.3 User APIs 27
9.4 Skill APIs 27
9.5 Request APIs 27
9.6 Session APIs 28
9.7 Review APIs 28
9.8 WebSocket Events (Chat) 28
10. Authentication and Authorization 29
10.1 JWT Authentication 29
10.2 Token Flow 29
10.3 Role-Based Access Control 29
10.4 Security Measures 29
11. Search and Filtering System 30
11.1 Search Architecture 30
11.2 Search Features 30
11.3 Search Query Example 30
11.4 Pagination Strategy 30
12. Real-Time Chat Architecture 31
12.1 WebSocket Infrastructure 31
12.2 Message Flow 31
12.3 Scalability Considerations 31
13. Frontend Architecture 32
13.1 Next.js Project Structure 32
13.2 State Management 32
13.3 API Integration 32
14. Scalability Design 33
14.1 Horizontal Scaling Strategy 33
14.2 Caching Strategy 33
14.3 Database Optimization 33

14.4 CDN and Asset Delivery 33

15. Security Design 34

15.1 Password Security 34

15.2 JWT Security 34

15.3 Rate Limiting 34

15.4 Input Validation 34

15.5 XSS and CSRF Protection 34

16. Testing Strategy 35

16.1 Testing Pyramid 35

16.2 Unit Testing 35

16.3 Integration Testing 35

16.4 End-to-End Testing 35

17. Deployment Architecture 36

17.1 Containerization 36

17.2 CI/CD Pipeline 36

17.3 Environment Strategy 36

18. Future Enhancements 37

18.1 AI-Powered Skill Matching 37

18.2 Integrated Video Calling 37

18.3 Skill Verification Badges 37

18.4 Mobile Application 37

18.5 Gamification 37

18.6 Community Features 37

19. Appendix 38

19.1 Example JSON Objects 38

User Object 38

Skill Object 38

19.2 API Response Format 38

19.3 HTTP Status Codes 38

19.4 Environment Variables 39

1. Introduction

1.1 Purpose of the Platform

SkillSwap is a peer-to-peer skill exchange platform designed to revolutionize how people learn and share knowledge. Unlike traditional learning platforms that require monetary payment, SkillSwap enables users to exchange skills directly with each other, creating a collaborative ecosystem where knowledge flows freely between community members.

The platform addresses the growing demand for accessible, affordable skill development by leveraging the untapped expertise within communities. Users can teach what they know and learn what they need, fostering a culture of continuous learning and mutual support.

1.2 Scope of the System

The SkillSwap platform encompasses a comprehensive set of features designed to facilitate seamless skill exchange:

- User profile management with skill listings
- Skill discovery and search functionality
- Skill exchange request system
- Real-time messaging between users
- Session scheduling and management
- Rating and review system
- Notification system for updates

1.3 Target Users

The platform is designed for a diverse range of users:

User Type	Description	Primary Use Case
Students	Individuals seeking to learn new skills	Find mentors for academic or career skills
Professionals	Working individuals with expertise	Share industry knowledge, learn adjacent skills
Hobbyists	Enthusiasts with specialized interests	Exchange craft, art, or recreational skills
Educators	Teachers and trainers	Expand teaching reach, learn new methodologies

User Type	Description	Primary Use Case
Career Changers	Individuals transitioning careers	Acquire new skills while sharing existing ones

1.4 Definitions and Terminology

Term	Definition
Skill	A specific competency or expertise that a user can teach or wants to learn
Skill Exchange	The mutual agreement between two users to teach each other different skills
Session	A scheduled meeting between users for teaching/learning a skill
Match	A pairing of users with complementary skill needs and offerings
Reputation Score	A calculated rating based on user reviews and exchange history
Availability	Time slots when a user is available for skill exchange sessions

2. Problem Statement

2.1 Real-World Problems

The current landscape of skill acquisition faces several significant challenges that limit access to knowledge and hinder personal growth:

Expensive Learning Platforms

Traditional online learning platforms charge substantial fees for courses, often ranging from $50 to $500 per course. This creates a significant barrier for individuals who cannot afford these costs, particularly students, career changers, and individuals in developing regions.

Lack of Peer-to-Peer Knowledge Exchange

While social media enables connection, there is no dedicated platform for structured skill exchange. People with valuable skills struggle to find those who want to learn them, and learners cannot easily identify willing teachers in their communities.

Limited Accessibility to Skill Learning

Geographic constraints, time zone differences, and lack of local expertise often make it difficult for individuals to access quality skill instruction. Traditional in-person learning requires physical presence, which is not always feasible.

Need for Collaborative Learning Ecosystems

Modern learners benefit from interactive, personalized instruction rather than passive video consumption. There is a growing need for platforms that facilitate active learning through direct human connection and mentorship.

2.2 How SkillSwap Solves These Problems

SkillSwap addresses these challenges through a comprehensive peer-to-peer exchange model:

Problem	Solution
High cost of learning	Barter-based exchange eliminates monetary barriers
Difficulty finding teachers/learners	Advanced matching algorithm connects complementary users
Geographic limitations	Virtual session support enables global connections
Passive learning experience	Interactive sessions with real-time feedback and

Problem	Solution
	mentorship
Lack of trust in online learning	Reputation system with verified reviews builds confidence

3. System Overview

3.1 Platform Architecture

SkillSwap is built on a modern, scalable microservices architecture designed to handle millions of users while maintaining high performance and reliability. The system is divided into logical modules that work together to provide a seamless user experience.

3.2 Major System Modules

Authentication System

Manages user identity, secure login, registration, and session management. Implements JWT-based authentication with support for OAuth providers (Google, GitHub) for streamlined onboarding.

User Profile Module

Handles user profile creation, management, and personalization. Includes avatar uploads, bio management, location settings, and skill preferences configuration.

Skill Management Module

Core functionality for creating, editing, and organizing skills. Supports categorization, tagging, proficiency levels, and detailed descriptions to help users showcase their expertise.

Skill Discovery Module

Enables users to find relevant skills and potential exchange partners through advanced search, filtering, and recommendation algorithms based on user preferences and behavior.

Exchange Request System

Manages the lifecycle of skill exchange requests, including creation, acceptance, rejection, and cancellation workflows with appropriate notifications.

Chat System

Real-time messaging infrastructure enabling users to communicate before, during, and after skill exchange sessions. Supports text, file sharing, and read receipts.

Session Scheduling Module

Calendar integration for scheduling skill exchange sessions, managing availability, sending reminders, and tracking session history.

Review & Rating System

Post-session feedback mechanism allowing users to rate and review their exchange partners, building a reputation system that promotes quality interactions.

Notification System

Multi-channel notification delivery (in-app, email, push) for important events such as new requests, messages, upcoming sessions, and system updates.

4. Functional Requirements

4.1 User Management

ID	Requirement	Priority
UM-001	Users shall be able to register with email/password or OAuth providers	High
UM-002	Users shall be able to log in securely with JWT authentication	High
UM-003	Users shall be able to reset forgotten passwords via email	High
UM-004	Users shall be able to update profile information including name, bio, avatar	High
UM-005	Users shall be able to set location preferences (city, timezone)	Medium
UM-006	Users shall be able to view their exchange history and statistics	Medium
UM-007	Users shall be able to deactivate or delete their account	Medium

4.2 Skill Management

ID	Requirement	Priority
SM-001	Users shall be able to add skills they can teach with details	High
SM-002	Users shall be able to add skills they want to learn	High
SM-003	Users shall be able to categorize skills (e.g., Technology, Arts, Languages)	High
SM-004	Users shall be able to specify proficiency level for each skill	High

ID	Requirement	Priority
SM-005	Users shall be able to add tags to skills for better discoverability	Medium
SM-006	Users shall be able to edit or remove existing skills	High
SM-007	Users shall be able to set availability for each teaching skill	Medium

4.3 Search & Filtering

ID	Requirement	Priority
SF-001	Users shall be able to search skills by keyword	High
SF-002	Users shall be able to filter by category, location, and skill level	High
SF-003	Users shall be able to filter by availability time slots	Medium
SF-004	Search results shall support pagination	High
SF-005	Users shall be able to sort results by relevance, rating, or recency	Medium

4.4 Exchange Requests

ID	Requirement	Priority
ER-001	Users shall be able to send skill exchange requests to other users	High
ER-002	Users shall be able to accept or reject incoming requests	High
ER-003	Users shall be able to include a message with their request	High
ER-004	Request status shall be tracked (pending, accepted, rejected, cancelled)	High

ID	Requirement	Priority
ER-005	Users shall be notified of request status changes	High
ER-006	Users shall be able to cancel pending requests	Medium

4.5 Session Scheduling

ID	Requirement	Priority
SS-001	Users shall be able to schedule sessions after request acceptance	High
SS-002	System shall send session reminders 24 hours and 1 hour before	High
SS-003	Users shall be able to reschedule or cancel sessions	High
SS-004	Sessions shall include video call links (integrated or external)	Medium
SS-005	Users shall be able to add session notes and resources	Medium
SS-006	Completed sessions shall be recorded in user history	High

4.6 Messaging

ID	Requirement	Priority
MSG-001	Users shall be able to send real-time text messages	High
MSG-002	Users shall be able to share files and images	Medium
MSG-003	Messages shall display read receipts	Medium
MSG-004	Message history shall be persisted and searchable	High

ID	Requirement	Priority
MSG-005	Users shall receive notifications for new messages	High

4.7 Notifications

ID	Requirement	Priority
NOT-001	Users shall receive in-app notifications for important events	High
NOT-002	Users shall receive email notifications for critical updates	High
NOT-003	Users shall be able to customize notification preferences	Medium
NOT-004	Notifications shall include request updates, messages, and session reminders	High

4.8 Ratings & Reviews

ID	Requirement	Priority
RR-001	Users shall be able to rate exchange partners after sessions	High
RR-002	Users shall be able to write detailed reviews	High
RR-003	Rating shall be on a scale of 1-5 stars	High
RR-004	Reviews shall be publicly visible on user profiles	High
RR-005	Users shall be able to respond to reviews	Low

5. Non-Functional Requirements

5.1 Scalability

The system must be designed to scale horizontally to accommodate growth:

- Support minimum 1 million registered users
- Handle 100,000 concurrent users during peak hours
- Process 10,000 requests per second at peak load
- Support 50,000 concurrent WebSocket connections for real-time chat
- Database must support 10 million+ documents per collection with query times under 100ms

5.2 Availability

Metric	Target	Measurement
Uptime	99.9%	Monthly availability excluding planned maintenance
Planned Downtime	< 4 hours/month	Scheduled during low-traffic periods
Recovery Time Objective (RTO)	< 15 minutes	Time to restore service after failure
Recovery Point Objective (RPO)	< 5 minutes	Maximum acceptable data loss

5.3 Performance

Operation	Target Response Time	Notes
Page Load	< 2 seconds	Initial page load with all assets
API Response	< 200ms	95th percentile for standard API calls
Search Query	< 500ms	Complex search with multiple filters
Message Delivery	< 100ms	Real-time message propagation
Image Upload	< 5 seconds	Standard profile image (up to 5MB)

5.4 Reliability

The system must maintain data integrity and consistent operation:

- All database writes must be atomic and support rollback on failure
- Message delivery must be guaranteed (at-least-once semantics)
- No data loss during normal operations or failover scenarios
- Automatic retry mechanisms for failed operations
- Circuit breakers to prevent cascade failures

5.5 Security

Requirement	Implementation
Data Encryption	TLS 1.3 for transit, AES-256 for data at rest
Password Security	bcrypt with salt rounds >= 12
Session Management	JWT with 24-hour expiration, secure httpOnly cookies
Input Validation	Server-side validation for all inputs, sanitization against XSS/SQL injection
Rate Limiting	100 requests per minute per IP, 1000 per authenticated user
Audit Logging	All authentication and sensitive operations logged

5.6 Maintainability

The codebase must support long-term maintenance and evolution:

- Modular architecture with clear separation of concerns
- Comprehensive API documentation with OpenAPI/Swagger
- Code coverage minimum 80% for unit tests
- CI/CD pipeline for automated testing and deployment
- Centralized logging and monitoring with structured logs

6. Technology Stack

6.1 Frontend Stack

Technology	Version	Purpose
Next.js	14.x	React framework with SSR/SSG for optimal performance
TypeScript	5.x	Type-safe JavaScript for maintainable code
TailwindCSS	3.x	Utility-first CSS framework for rapid styling
React Query	5.x	Server state management and caching
Zustand	4.x	Client state management for UI state
Socket.io Client	4.x	Real-time communication for chat
React Hook Form	7.x	Form handling with validation
Zod	3.x	Schema validation for TypeScript

6.2 Backend Stack

Technology	Version	Purpose
Node.js	20.x LTS	JavaScript runtime for server-side execution
Express.js	4.x	Web framework for REST API development
TypeScript	5.x	Type safety for backend services
Socket.io	4.x	WebSocket server for real-time features
JWT	9.x	Token-based authentication
bcrypt	5.x	Password hashing
Helmet	7.x	Security headers middleware

Technology	Version	Purpose
CORS	2.x	Cross-origin resource sharing configuration

6.3 Database & Storage

Technology	Version	Purpose
MongoDB	7.x	Primary document database for flexible schema
Mongoose	8.x	ODM for MongoDB with schema validation
Redis	7.x	In-memory cache and session store
Cloudinary	Latest	Image upload and transformation service
AWS S3	Latest	File storage for documents and backups

6.4 Infrastructure

Technology	Purpose
Docker	Containerization for consistent deployments
Kubernetes	Container orchestration for scaling
AWS ECS/EKS	Managed container service hosting
AWS RDS	Managed database hosting (if needed)
AWS ElastiCache	Managed Redis hosting
AWS CloudFront	CDN for static assets
AWS Route 53	DNS management
GitHub Actions	CI/CD pipeline automation

6.5 Technology Selection Rationale

The technology stack was selected based on the following criteria:

- JavaScript/TypeScript ecosystem enables full-stack development with shared types and utilities
- MongoDB's flexible schema accommodates evolving data models without migrations
- Next.js provides excellent developer experience with built-in optimizations
- Redis enables sub-millisecond caching for frequently accessed data
- AWS services provide enterprise-grade reliability and scalability

7. High-Level System Architecture

7.1 Architecture Overview

SkillSwap follows a microservices architecture pattern with clear separation of concerns. The system is designed to scale horizontally, allowing individual services to be scaled independently based on demand.

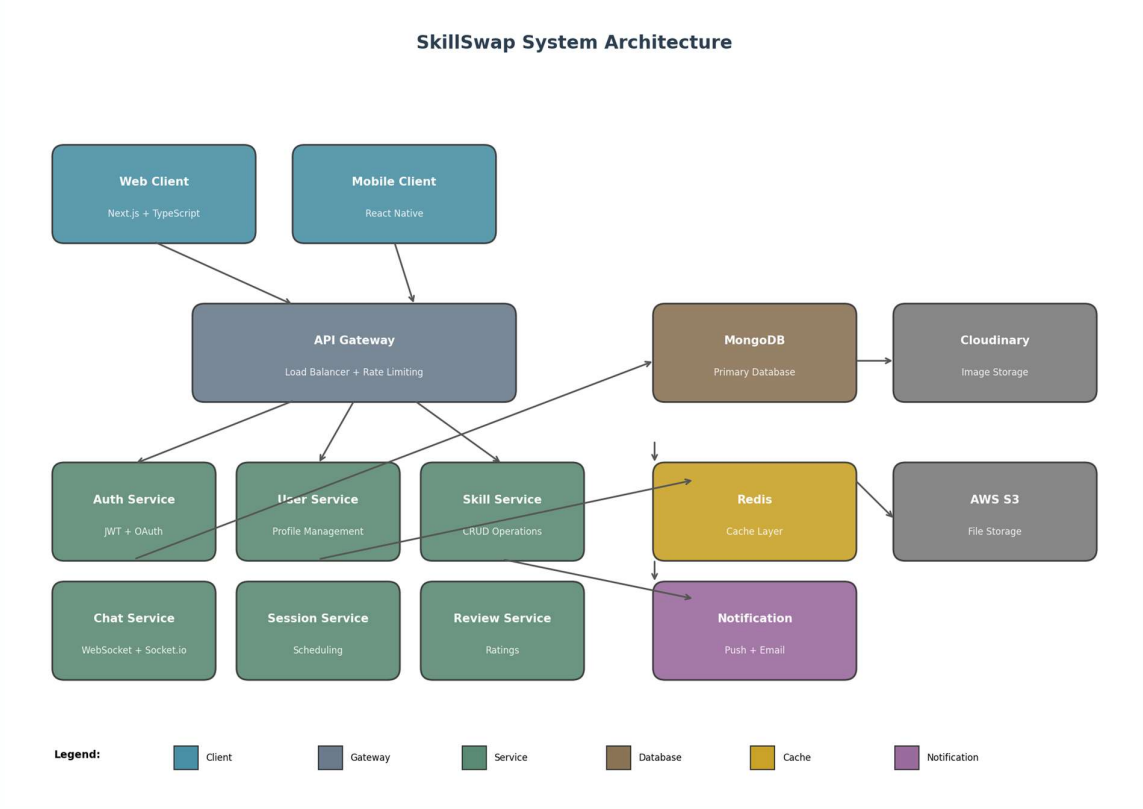

Figure 1: SkillSwap System Architecture Diagram

7.2 Component Description

Client Layer

The client layer consists of web applications built with Next.js and potential mobile applications. These clients communicate with the backend via REST APIs and WebSocket connections for real-time features.

API Gateway

The API Gateway serves as the single entry point for all client requests. It handles authentication validation, rate limiting, request routing, and load balancing across backend services.

Microservices Layer

The backend is organized into domain-specific microservices:

- Auth Service: Handles user authentication, token generation, and OAuth integration
- User Service: Manages user profiles, preferences, and account settings
- Skill Service: CRUD operations for skills, categories, and tags
- Request Service: Manages skill exchange request lifecycle
- Chat Service: Real-time messaging using WebSocket connections
- Session Service: Scheduling, reminders, and session management
- Review Service: Rating and review collection and aggregation
- Notification Service: Multi-channel notification delivery

Data Layer

The data layer consists of MongoDB as the primary database, Redis for caching and session storage, and external services for file storage (Cloudinary for images, AWS S3 for documents).

7.3 Data Flow

A typical request flows through the system as follows:

0. Client sends request to API Gateway with JWT token
1. Gateway validates token and applies rate limiting
2. Request is routed to appropriate microservice
3. Service checks Redis cache for frequently accessed data
4. If cache miss, service queries MongoDB
5. Response is cached (if applicable) and returned to client

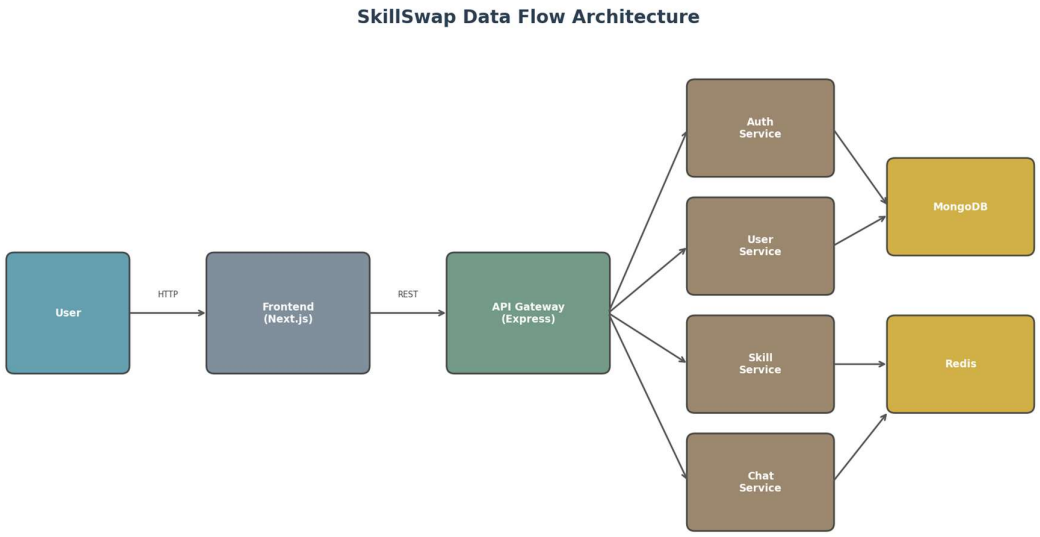

Figure 2: SkillSwap Data Flow Diagram

8. Database Design

8.1 Database Selection

MongoDB was selected as the primary database due to its flexible schema design, horizontal scalability, and excellent support for complex queries with its aggregation framework. The document-oriented nature aligns well with the varied data structures in SkillSwap.

8.2 Collection Schema

Users Collection

```
{ _id: ObjectId, email: String (unique, indexed), password: String (hashed),
name: String, bio: String, avatar: String (URL), location: { city: String,
country: String, timezone: String }, reputationScore: Number (default: 0),
totalExchanges: Number (default: 0), isVerified: Boolean (default: false),
createdAt: Date, updatedAt: Date }
```

Skills Collection

```
{ _id: ObjectId, userId: ObjectId (ref: Users, indexed), name: String,
category: String (indexed), description: String, level: String (enum: ['Beginner',
'Intermediate', 'Advanced', 'Expert']), isTeaching: Boolean (indexed), tags:
[String], availability: [{ day: String, startTime: String, endTime:
String }], createdAt: Date }
```

SkillRequests Collection

```
{ _id: ObjectId, senderId: ObjectId (ref: Users, indexed), receiverId: ObjectId
(ref: Users, indexed), skillOfferedId: ObjectId (ref: Skills), skillWantedId:
ObjectId (ref: Skills), status: String (enum: ['Pending', 'Accepted', 'Rejected',
'Cancelled'], indexed), message: String, createdAt: Date, updatedAt: Date }
```

Sessions Collection

```
{ _id: ObjectId, requestId: ObjectId (ref: SkillRequests), teacherId: ObjectId
(ref: Users), studentId: ObjectId (ref: Users), scheduledAt: Date (indexed),
duration: Number (minutes), status: String (enum: ['Scheduled', 'Completed',
'Cancelled', 'NoShow']), meetingLink: String, notes: String, createdAt: Date }
```

Messages Collection

```
{ _id: ObjectId, senderId: ObjectId (ref: Users, indexed), receiverId: ObjectId
(ref: Users, indexed), content: String, type: String (enum: ['Text', 'Image',
'File']), isRead: Boolean (default: false), createdAt: Date (indexed) }
```

Reviews Collection

```
{ _id: ObjectId, sessionId: ObjectId (ref: Sessions), reviewerId: ObjectId (ref:
Users), revieweeId: ObjectId (ref: Users, indexed), rating: Number (1-5),
comment: String, createdAt: Date }
```

Notifications Collection

```
{   _id: ObjectId,   userId: ObjectId (ref: Users, indexed),   type: String (enum:
['Request', 'Message', 'Session', 'System']),   title: String,   message: String,
isRead: Boolean (default: false),   data: Object,   createdAt: Date (indexed) }
```

8.3 Database Indexes

Collection	Index Fields	Type	Purpose
Users	email	Unique	Fast login lookup
Skills	userId	Single	Find user's skills
Skills	category, isTeaching	Compound	Skill discovery queries
Skills	tags	Multikey	Tag-based search
SkillRequests	senderId, status	Compound	User's sent requests
SkillRequests	receiverId, status	Compound	User's received requests
Messages	senderId, receiverId, createdAt	Compound	Conversation history
Sessions	teacherId, scheduledAt	Compound	Teacher's schedule
Sessions	studentId, scheduledAt	Compound	Student's schedule
Reviews	revieweeId	Single	User's reputation calculation

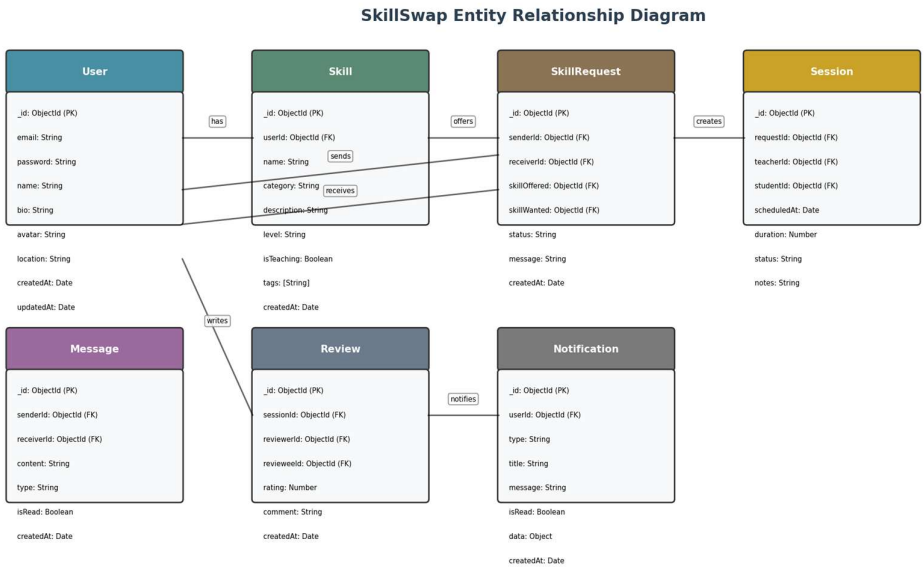

Figure 3: SkillSwap Entity Relationship Diagram

9. API Design

9.1 API Overview

SkillSwap exposes a RESTful API following standard HTTP conventions. All endpoints return JSON responses and use appropriate HTTP status codes. Authentication is required for most endpoints via Bearer token in the Authorization header.

9.2 Authentication APIs

```
POST /api/auth/register Request: {   "email": "user@example.com",   "password": "securePassword123",   "name": "John Doe" }  Response (201): {   "success": true, "data": {     "user": { ... },     "token": "eyJhbGciOiJIUzI1NiIs..."   } }

POST /api/auth/login Request: {   "email": "user@example.com",   "password": "securePassword123" }  Response (200): {   "success": true,   "data": {     "user": { ... },     "token": "eyJhbGciOiJIUzI1NiIs..."   } }
```

9.3 User APIs

```
GET /api/users/profile Headers: Authorization: Bearer <token> Response (200): { "success": true,   "data": {     "_id": "...",     "email": "user@example.com", "name": "John Doe",     "bio": "Full-stack developer...",     "avatar": "https://...", "skillsTeaching": [...],     "skillsLearning": [...]   } }

PUT /api/users/profile Headers: Authorization: Bearer <token> Request: {   "name": "John Updated",   "bio": "Updated bio...",   "location": { "city": "New York", "country": "USA" } } Response (200): { "success": true, "data": { ... } }
```

9.4 Skill APIs

```
POST /api/skills Headers: Authorization: Bearer <token> Request: {   "name": "React.js",   "category": "Technology",   "description": "Modern frontend library...", "level": "Expert",   "isTeaching": true,   "tags": ["javascript", "frontend"] } Response (201): { "success": true, "data": { "_id": "...", ... } }

GET /api/skills/search?query=react&category=Technology&page=1&limit=20 Response (200): {   "success": true,   "data": {     "skills": [...],     "pagination": { "page": 1, "limit": 20, "total": 150, "totalPages": 8 }   } }
```

9.5 Request APIs

```
POST /api/requests Headers: Authorization: Bearer <token> Request: {   "receiverId": "userId123",   "skillOfferedId": "skillId456",   "skillWantedId": "skillId789", "message": "I'd love to exchange skills with you!" } Response (201): { "success": true, "data": { "_id": "...", "status": "Pending", ... } }

PUT /api/requests/:id/accept Headers: Authorization: Bearer <token> Response (200): { "success": true, "data": { "status": "Accepted", ... } }
```

```
PUT /api/requests/:id/reject Headers: Authorization: Bearer <token> Request: {
"reason": "Not available at the moment" } Response (200): { "success": true, "data": {
"status": "Rejected", ... } }
```

9.6 Session APIs

```
POST /api/sessions Headers: Authorization: Bearer <token> Request: {   "requestId":
"requestId123",   "scheduledAt": "2026-05-15T14:00:00Z",   "duration": 60,   "notes":
"We'll cover React hooks and context" } Response (201): { "success": true, "data": {
"_id": "...", ... } }
```

9.7 Review APIs

```
POST /api/reviews Headers: Authorization: Bearer <token> Request: {   "sessionId":
"sessionId123",   "revieweeId": "userId456",   "rating": 5,   "comment": "Excellent
teacher, very patient and knowledgeable!" } Response (201): { "success": true, "data":
{ "_id": "...", ... } }
```

9.8 WebSocket Events (Chat)

```
// Client connects to: wss://api.skillswap.com/socket?token=<jwt>  // Send message
emit('message:send', {   receiverId: 'userId123',   content: 'Hello! Are you available
for a session?',   type: 'Text' })  // Receive message on('message:received', (data) =>
{   // { _id, senderId, content, createdAt, ... } })  // Mark as read
emit('message:read', { messageId: 'msgId123' })
```

10. Authentication and Authorization

10.1 JWT Authentication

SkillSwap uses JSON Web Tokens (JWT) for stateless authentication. The token contains the user's ID and role, signed with a server-side secret key.

```
JWT Payload Structure: {   "sub": "userId123",      // Subject (user ID)   "email":
"user@example.com",   "role": "user",   "iat": 1714502400,       // Issued at   "exp":
1714588800        // Expiration (24 hours) }
```

10.2 Token Flow

6. User logs in with credentials
7. Server validates credentials and generates JWT
8. Token is returned to client and stored (httpOnly cookie or secure storage)
9. Client includes token in Authorization header for subsequent requests
10. Server validates token signature and expiration on each request

10.3 Role-Based Access Control

Role	Permissions
User	Manage own profile, skills, requests, sessions, messages
Moderator	User permissions + review reports, suspend users
Admin	Full system access, user management, analytics

10.4 Security Measures

- Tokens expire after 24 hours requiring re-authentication
- Refresh token mechanism for seamless user experience
- Passwords hashed with bcrypt (12+ salt rounds)
- Rate limiting on authentication endpoints (5 attempts per 15 minutes)
- Account lockout after 5 failed login attempts

11. Search and Filtering System

11.1 Search Architecture

The search system uses MongoDB's text indexes and aggregation pipeline for efficient querying. For production scale, Elasticsearch integration is recommended for advanced full-text search capabilities.

11.2 Search Features

Feature	Implementation
Keyword Search	MongoDB text index on skill name and description
Category Filter	Exact match on category field with index
Skill Level Filter	Enum match on level field
Location Filter	Geospatial query for nearby users
Availability Filter	Match user's available time slots
Tag Filter	Multikey index for array matching

11.3 Search Query Example

```
// MongoDB aggregation pipeline for skill search db.skills.aggregate([   { $match: {
$text: { $search: "react" } } },   { $match: { category: "Technology", isTeaching: true
} },   { $lookup: {       from: "users",       localField: "userId",
foreignField: "_id",       as: "teacher"   }},   { $sort: { score: { $meta: "textScore"
} } },   { $skip: 0 },   { $limit: 20 } ])
```

11.4 Pagination Strategy

Search results use cursor-based pagination for optimal performance with large datasets. Each page returns a cursor token for fetching the next page, avoiding offset-based performance degradation.

12. Real-Time Chat Architecture

12.1 WebSocket Infrastructure

The chat system uses Socket.io for real-time bidirectional communication. It provides automatic reconnection, room-based messaging, and fallback to HTTP long-polling for older browsers.

12.2 Message Flow

11. Client establishes WebSocket connection with JWT token
12. Server authenticates connection and joins user to personal room
13. Sender emits 'message:send' event with receiver ID and content
14. Server validates message and persists to MongoDB
15. Server emits 'message:received' to receiver's room
16. Receiver acknowledges receipt, server updates 'isRead' status

12.3 Scalability Considerations

Challenge	Solution
Horizontal scaling	Redis Adapter for Socket.io enables multi-server communication
Message persistence	All messages stored in MongoDB with TTL for cleanup
Offline delivery	Undelivered messages queued and sent on reconnection
Typing indicators	Throttled events (300ms debounce) to reduce noise

13. Frontend Architecture

13.1 Next.js Project Structure

```
src/ ├── app/                    # Next.js App Router │   ├── (auth)/           # Auth
route group │   │   ├── login/ │   │   ├── register/ │   ├── (main)/           # Main
route group │   │   ├── dashboard/ │   │   ├── profile/ │   │   ├── skills/ │   │   ├──
messages/ │   ├── api/              # API routes (if needed) │   ├── layout.tsx │
└── page.tsx ├── components/           # React components │   ├── ui/              #
Reusable UI components │   ├── forms/           # Form components │   ├── layout/
# Layout components │   └── features/        # Feature-specific components ├── hooks/
# Custom React hooks ├── lib/                 # Utilities and configurations │   ├──
api.ts          # API client │   ├── socket.ts        # Socket.io client │   └──
utils.ts ├── stores/              # Zustand state stores ├── types/               #
TypeScript type definitions └── styles/              # Global styles
```

13.2 State Management

The frontend uses a hybrid state management approach:

- React Query: Server state (API data, caching, synchronization)
- Zustand: Client state (UI state, user preferences)
- Socket.io: Real-time state (messages, notifications)

13.3 API Integration

```
// Example API client with React Query import { useQuery, useMutation } from
'@tanstack/react-query'; import { api } from '@/lib/api';  // Fetch user profile export
const useProfile = () => {   return useQuery({     queryKey: ['profile'],     queryFn:
() => api.get('/users/profile'),   }); };  // Update profile mutation export const
useUpdateProfile = () => {   return useMutation({     mutationFn: (data) =>
api.put('/users/profile', data),     onSuccess: () => {
queryClient.invalidateQueries({ queryKey: ['profile'] });     },   }); };
```

14. Scalability Design

14.1 Horizontal Scaling Strategy

The system is designed to scale horizontally by adding more server instances as demand increases:

- Stateless microservices allow easy replication
- Load balancer distributes traffic across instances
- Auto-scaling based on CPU/memory thresholds
- Database sharding for horizontal data distribution

14.2 Caching Strategy

Cache Layer	Data	TTL
Browser Cache	Static assets, images	1 year
CDN (CloudFront)	Global static content	24 hours
Redis	User sessions, frequent queries	1 hour
Application Cache	Skill categories, popular searches	15 minutes

14.3 Database Optimization

- Strategic indexing on frequently queried fields
- Compound indexes for multi-field queries
- Read replicas for read-heavy workloads
- Connection pooling (max 100 connections per instance)

14.4 CDN and Asset Delivery

Static assets are served through AWS CloudFront CDN with edge caching. User-uploaded images are processed and optimized by Cloudinary before delivery.

15. Security Design

15.1 Password Security

User passwords are hashed using bcrypt with a minimum of 12 salt rounds. This provides strong protection against brute-force attacks and rainbow table attacks.

15.2 JWT Security

- Short-lived access tokens (24 hours)
- Refresh tokens with rotation (7 days)
- Tokens signed with HS256 algorithm and strong secret
- Secure httpOnly cookies for web clients

15.3 Rate Limiting

Endpoint	Limit	Window
Authentication	5 requests	15 minutes
API (authenticated)	1000 requests	15 minutes
API (unauthenticated)	100 requests	15 minutes
File uploads	10 uploads	1 hour

15.4 Input Validation

All user inputs are validated using Zod schemas on both client and server. This prevents injection attacks and ensures data integrity.

15.5 XSS and CSRF Protection

- Helmet.js middleware sets security headers
- Content Security Policy restricts resource loading
- CSRF tokens for state-changing operations
- Output encoding to prevent XSS

16. Testing Strategy

16.1 Testing Pyramid

The testing strategy follows the testing pyramid with a focus on unit tests, supplemented by integration and E2E tests:

- Unit Tests: 70% - Fast, isolated component testing
- Integration Tests: 20% - API and service integration
- E2E Tests: 10% - Critical user journey validation

16.2 Unit Testing

Component	Tool	Coverage Target
Frontend Components	Jest + React Testing Library	80%
API Handlers	Jest + Supertest	85%
Services	Jest	90%
Utilities	Jest	95%

16.3 Integration Testing

Integration tests verify that different parts of the system work together correctly. These tests use a test database and mock external services.

16.4 End-to-End Testing

E2E tests validate critical user flows using Playwright. Tests run against a staging environment that mirrors production.

```
// Example E2E test with Playwright test('user can complete skill exchange flow', async ({ page }) => {   await page.goto('/login');   await page.fill('[name=email]', 'test@example.com');   await page.fill('[name=password]', 'password');   await page.click('button[type=submit]');      await page.waitForURL('/dashboard');   await page.click('text=Find Skills');   await page.fill('[name=search]', 'react');   await page.click('text=Send Request');      await expect(page.locator('text=Request sent')).toBeVisible(); });
```

17. Deployment Architecture

17.1 Containerization

All services are containerized using Docker for consistent deployments across environments. Docker Compose is used for local development, while Kubernetes manages production deployments.

```
# Example Dockerfile for API service FROM node:20-alpine WORKDIR /app COPY
package*.json ./ RUN npm ci --only=production COPY . . EXPOSE 3000 CMD ["node",
"dist/index.js"]
```

17.2 CI/CD Pipeline

GitHub Actions automates the CI/CD pipeline:

17. Code push triggers automated tests
18. On test success, Docker image is built and pushed to ECR
19. Kubernetes deployment is updated with new image
20. Health checks verify successful deployment
21. Rollback available if health checks fail

17.3 Environment Strategy

Environment	Purpose	Infrastructure
Development	Local development	Docker Compose
Staging	Pre-production testing	AWS ECS (small instances)
Production	Live application	AWS EKS with auto-scaling

18. Future Enhancements

18.1 AI-Powered Skill Matching

Implement machine learning algorithms to analyze user behavior, skill preferences, and learning patterns to suggest optimal skill exchange matches. The system would consider factors like learning style compatibility, timezone overlap, and historical exchange success rates.

18.2 Integrated Video Calling

Build native video calling functionality using WebRTC, eliminating the need for external meeting links. Features would include screen sharing, virtual whiteboards, and session recording for review.

18.3 Skill Verification Badges

Implement a verification system where users can earn badges by completing assessments or providing credentials. Verified skills would display a badge icon, increasing trust in the platform.

18.4 Mobile Application

Develop native mobile applications for iOS and Android using React Native. Mobile apps would provide push notifications, camera integration for skill demonstrations, and offline message queuing.

18.5 Gamification

Introduce gamification elements to increase user engagement:

- Achievement badges for milestones (first exchange, 10 sessions, etc.)
- Leaderboards for top contributors and highest-rated teachers
- Skill points system for completed exchanges
- Streak tracking for consistent learning

18.6 Community Features

Expand beyond one-on-one exchanges with community features:

- Group study sessions for multiple learners
- Discussion forums for each skill category
- Resource sharing (articles, videos, tutorials)
- Local meetup organization

19. Appendix

19.1 Example JSON Objects

User Object

```
{   "_id": "6628a1b2c3d4e5f6g7h8i9j0",   "email": "jane.doe@example.com",   "name":
"Jane Doe",   "bio": "Full-stack developer passionate about teaching React",
"avatar": "https://res.cloudinary.com/skillswap/avatars/jane.jpg",   "location": {
"city": "San Francisco",     "country": "USA",     "timezone": "America/Los_Angeles"
},   "reputationScore": 4.8,   "totalExchanges": 23,   "isVerified": true,
"createdAt": "2026-01-15T08:30:00Z",   "updatedAt": "2026-04-10T14:22:00Z" }
```

Skill Object

```
{   "_id": "7729b2c3d4e5f6g7h8i9j0k1",   "userId": "6628a1b2c3d4e5f6g7h8i9j0",
"name": "React.js Development",   "category": "Technology",   "description": "Learn
modern React with hooks, context, and best practices",   "level": "Expert",
"isTeaching": true,   "tags": ["javascript", "frontend", "web-development"],
"availability": [     { "day": "Monday", "startTime": "18:00", "endTime": "21:00" },
{ "day": "Saturday", "startTime": "10:00", "endTime": "16:00" }   ],   "createdAt":
"2026-01-20T10:15:00Z" }
```

19.2 API Response Format

```
// Success Response {   "success": true,   "data": { ... },   "meta": {     "page": 1,
"limit": 20,     "total": 150   } }  // Error Response {   "success": false,   "error":
{     "code": "VALIDATION_ERROR",     "message": "Invalid input data",     "details": [
{ "field": "email", "message": "Email is required" }     ]   } }
```

19.3 HTTP Status Codes

Status Code	Meaning	Usage
200 OK	Success	GET, PUT, DELETE operations
201 Created	Resource created	POST operations
400 Bad Request	Invalid input	Validation errors
401 Unauthorized	Not authenticated	Missing/invalid token
403 Forbidden	Not authorized	Insufficient permissions
404 Not Found	Resource not found	Invalid ID or route
429 Too Many Requests	Rate limited	Exceeded rate limit

Status Code	Meaning	Usage
500 Server Error	Internal error	Unexpected server error

19.4 Environment Variables

```
# Server Configuration PORT=3000 NODE_ENV=production  # Database
MONGODB_URI=mongodb://... REDIS_URL=redis://...  # Authentication JWT_SECRET=your-
secret-key JWT_EXPIRES_IN=24h  # External Services CLOUDINARY_CLOUD_NAME=...
CLOUDINARY_API_KEY=... CLOUDINARY_API_SECRET=...  AWS_ACCESS_KEY_ID=...
AWS_SECRET_ACCESS_KEY=...
```

SkillSwap

Learn. Teach. Grow Together.

contact@skillswap.io | www.skillswap.io

© 2026 SkillSwap Platform. All rights reserved.